SCCAD 3nm PDK

User Manual

Physical Design with Cadence Innovus

Version	v1.0
Release Date	TBD
Organization	SCCAD Lab, University of Southern California
Tool	Cadence Innovus
Contact	sw.jung@gatech.edu (Sungwoo Jung)

This manual provides comprehensive guidance for users working with the **SCCAD 3nm Process Design Kit (PDK)**. It covers PDK installation from GitHub, environment configuration, and step-by-step instructions for running Place-and-Route (PnR) using **Cadence Innovus**.

1 Introduction

1.1 Overview of SCCAD 3nm PDK

The SCCAD 3nm Process Design Kit (PDK) is developed and maintained by the SCCAD Laboratory to support academic and research-oriented digital IC design flows at the 3 nm technology node. The PDK provides a complete set of design enablement files including technology files, standard-cell libraries, SPICE models, and interconnect models. This allow designers to perform synthesis, place-and-route (PnR), and sign-off verification using industry-standard EDA tools.

Several open-source 3 nm PDKs are available to the research community. The table below positions SCCAD-3nm alongside two other academic PDKs to highlight the distinguishing capabilities of this release.

Feature	Academic PDK A	Academic PDK B	SCCAD-3nm
Tech Node	3nm	3nm	**3nm**
Device Type	GAAFET	GAAFET	**GAAFET**
Cell Heights	5.5T	6T	**6T, 5T**
Power Rail (PR)	Front-side (FSPR)	Buried (BPR)	**FSPR, BPR**
Support Back-side Metal	No	No	**Yes**

As shown in the table, all three PDKs target the 3 nm node and adopt Gate-All-Around FET (GAAFET) device architectures. Within this shared foundation, the SCCAD-3nm PDK offers several capabilities that extend the design space available to researchers.

- **Dual cell-height support (6T and 5T).** Providing both 6-track and 5-track standard-cell libraries gives designers flexibility to trade off area density against routability.
- **Both Front-side and Buried Power Rail (FSPR & BPR).** Supporting both power delivery architectures allows direct comparison of conventional front-side routing against buried power rail approaches.
- **Back-side metal support.** SCCAD-3nm includes back-side metal interconnect, enabling exploration of back-side power delivery network (BS-PDN) and signal routing techniques.

1.2 Key Features and Technology Highlights

Technology Node: 3 nm GAAFET

The SCCAD-3nm PDK is built on a **3 nm Gate-All-Around FET (GAAFET)** process. Unlike FinFET, where the gate wraps around three sides of a fin, GAAFET surrounds the channel on all four sides using stacked nanosheet or nanowire structures. This architecture provides superior electrostatic control, reduced short-channel effects, and improved drive current at scaled supply voltages. The fabrication process sequence is illustrated in Figure 1.

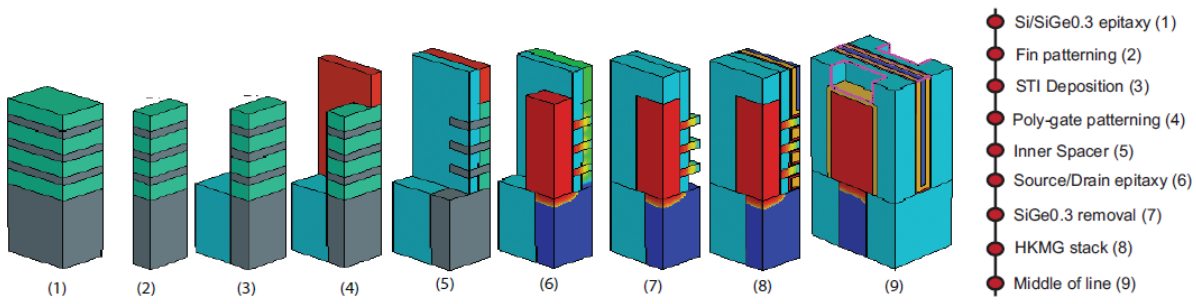


Table 1 summarizes the key device physical parameters of the SCCAD-3nm NSFET device in comparison with a representative 5 nm FinFET process.

Table 1. Comparison of device physical parameters of FinFET and NSFET device.

Physical Parameters (nm)	TSMC 5nm	NSFET 3nm
CPP	48	**42**
Fin/NS pitch	25	**NA**
Gate length	15	**12**
Spacer length	6	**5**
# Fin/NS	2	**3**
Fin/NS thickness	6	**5**
Fin/NS width	NA	**30, 20**
Fin/NS stack height	55	**55**

Standard-Cell Library

The SCCAD-3nm PDK includes two standard-cell libraries. The **6-Track (6T) library** uses a Front-side Power Rail (FSPR) architecture, and the **5-Track (5T) library** adopts a Buried Power Rail (BPR) architecture. The complete cell set comprises **57 standard cells**. Table 2 lists the full cell set along with drive strength variants for each cell type.

Table 2. Standard-cell library overview.

STD Cells	(Input) x (# parallel Trs)
INV	x1, x2, x3, x4, x5, x6, x7, x8, x10, x12, x14, x16
BUF	x1, x2, x3, x4, x5, x6, x7, x8, x10
NAND, NOR	2x1, 3x1, 4x1, 2x2, 3x2
AND, OR	2x1, 3x1, 4x1
XOR, XNOR	2x1, 3x1
AOI, OAI	(21, 22, 221, 222, 311, 31) x1
MUX	2x1
D Flip-Flop	Hx1 (async.), HQNx1 (sync.)
DHL (latch)	x1
Total	**57**

Each cell type in the library serves a distinct role in digital circuit design:

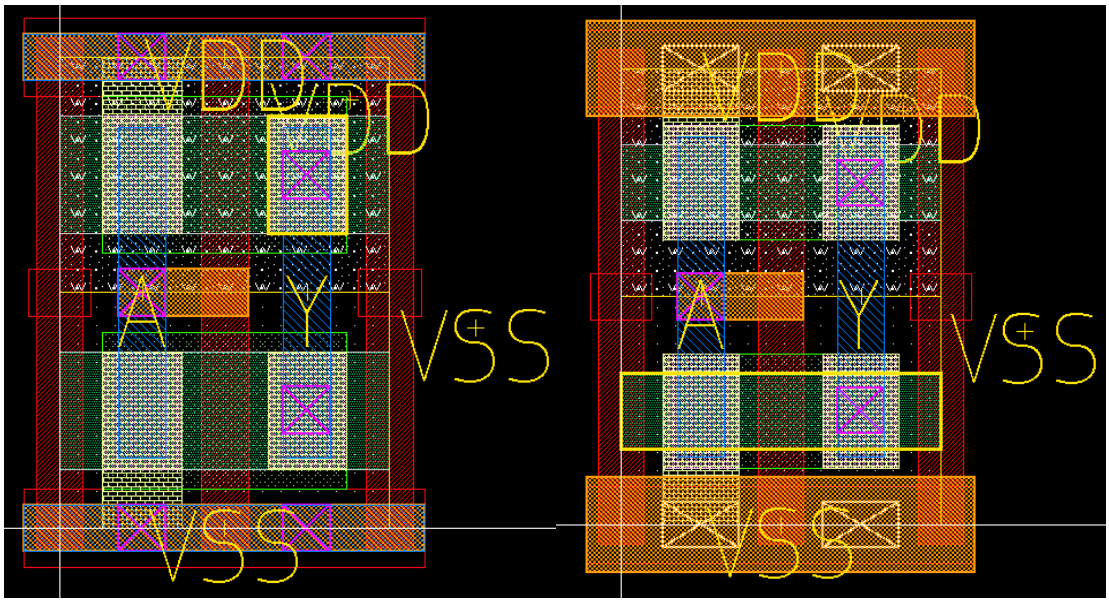
- **INV / BUF** — Inverter and buffer cells provided in 12 and 9 drive strength variants, forming the backbone of clock tree synthesis and signal-integrity-driven buffering.
- **NAND / NOR** — 2-, 3-, and 4-input gates supporting both area-optimized and performance-optimized implementations.
- **AND / OR** — Non-inverting gates in 2-, 3-, and 4-input configurations.
- **XOR / XNOR** — Essential for arithmetic datapaths, parity generation, and error-detection logic.

- **AOI / OAI** — Complex gates in six configurations enabling compact, high-speed multi-level boolean function implementation.
- **MUX** — A 2-to-1 multiplexer for datapath and control-flow selection.
- **D Flip-Flop** — Asynchronous reset type (Hx1) and synchronous reset type with complementary output (HQNx1).
- **DHL (Latch)** — A level-sensitive D-latch for time-borrowing and latch-based pipeline designs.

Figure 2 shows the layout of the **INVx1** cell for each library variant.

6-Track (6T) / Front-Side Power Rail (FSPR): In the 6T library, the power rails (VDD and VSS) are routed on the M1 metal layer, each rail being 3x the Critical Dimension (CD) wide. The resulting cell height is 144 nm. The nanosheet width is fixed at 30 nm.

5-Track (5T) / Buried Power Rail (BPR): In the 5T library, the power rails are moved to the buried metal layer MBPR, connected through VBPR vias (12 nm x 18 nm). The MBPR metal width is 25 nm. The effective cell height is reduced to 120 nm.



[FIGURE 2a/b: INVx1 layout of 6T FSPR and 5T BPR library]

Back-end-of-Line (BEOL) Technology

The SCCAD-3nm PDK features a comprehensive BEOL stack from **MB2 through M9** and via layers **VB1 through V8**. Table 3 summarizes the key physical parameters.

Table 3. BEOL Metal and Via Layers — Physical Parameters.

Tier	Metal	W (nm)	Resistance (Ω)
Back-side	MB2-MB1	61	11
BPR cell layer	MBPR	25	65
Front-Side	M0	20	523
	M1-M3	12	347
	M4, M5	18	101
	M6, M7	24	44

Tier	VIA	W x L (nm²)	Resistance (Ω)
Back-Side	VB1	40 x 40	4.0
BPR cell layer	VBPR	20 x 12	56
BSP cell layer	µ-TSV	60 x 60	5
	BSC	20 x 20	74.9
Front-Side	V0-V3	12 x 12	63.5
	V4, V5	18 x 18	19.38
	V6, V7	24 x 24	10.8

Among the layers listed above, the SCCAD-3nm PDK provides dedicated back-side metal interconnect through two layers: **MB1** and **MB2**. These layers are available for both Power Delivery Network (PDN) routing and general signal routing.

Using MB1 and MB2 for PDN distribution — commonly referred to as a **Back-Side Power Delivery Network (BS-PDN)** — decouples power routing from signal routing, reducing IR drop and freeing front-side metal resources for timing-critical signals.

EDA Tool Support

The SCCAD-3nm PDK is validated with the following industry-standard EDA tools. Two separate user manuals are provided — one for the Synopsys ICC2 flow (this document) and one for the Cadence Innovus flow.

2 Getting Started — PDK Installation from GitHub

2.1 Cloning the Repository

The SCCAD 3nm PDK is hosted on GitHub. Use the following commands to clone the repository to your local workstation or server.

HTTPS:

```
git clone https://github.com/sjung366/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

SSH (recommended for development):

```
git clone git@github.com:SCCAD-Lab/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

If the repository uses Git submodules, initialize them as follows:

```
git submodule update --init --recursive
```

2.2 Repository Structure Walkthrough

The repository is organized into three top-level directories: PDK-Synopsys, PDK-Cadence, and Sample Designs. This manual covers the PDK-Cadence flow. Each version (Front-side / Back-side) has an identical folder structure.

```
SCCAD-3nm-PDK/
```

```
├── PDK-Cadence/
│   ├── Front-side Version/
│   │   ├── PDK Development/
│   │   │   ├── DRC/            # DRC rule decks
│   │   │   ├── LVS_PEX/        # LVS/PEX decks
│   │   │   ├── MODEL/          # HSPICE SPICE model cards
│   │   │   ├── SCRIPTS/        # PDK build and utility scripts
│   │   │   └── STD-CELLS/      # Standard cell GDS layouts
│   │   └── PnR/
│   │       ├── TF/             # Technology file
│   │       ├── TECH-LEF/       # Technology LEF
│   │       ├── LEF/            # Cell LEF
│   │       ├── LIB_DB/         # Liberty (.lib) files
│   │       └── QRC/            # QRC technology file for Quantus
│   └── Back-side Version/      # Identical structure (BPR variant)
├── PDK-Synopsys/
│   ├── Front-side Version/     # Identical structure, ITF instead of QRC
│   └── Back-side Version/
└── Sample Designs/
    ├── ecg/
    │   ├── ecg.netlist.v
    │   ├── ecg.sdc
    │   └── ecg_final_design.def
    └── openpiton/
        ├── HARD-LEF (OpenPiton)/
        ├── tile.netlist.v
        ├── tile.sdc
        ├── openpiton_final_design.def
        └── Layouts of sample designs.png
```

The table below describes the role of each key directory under PDK-Cadence/Front-side Version/PnR/:

Directory	Description
PnR/TECH-LEF/	Technology LEF defining layer names, routing directions, and spacing rules — loaded directly by Innovus
PnR/LEF/	Cell LEF files providing macro abstracts (pin locations, obstructions, cell dimensions)
PnR/LIB_DB/	Liberty (.lib) files with timing arcs and power tables for Innovus and Tempus
PnR/QRC/	QRC technology file (.tch) — Quantus-native RC model for full-chip parasitic extraction
PDK Development/MODEL/	HSPICE SPICE model cards for NMOS/PMOS across all PVT corners
PDK Development/DRC/	DRC rule decks for IC Validator or compatible tool
PDK Development/LVS_PEX/	LVS decks and parasitic extraction configuration
Sample Designs/	ECG and OpenPiton benchmark designs with netlist, SDC, and final DEF

2.3 Environment Setup

Before running any EDA tool, configure the following environment variables in your shell environment (e.g., **.bashrc** or **.cshrc**).

Required Environment Variables (Front-side Version):

```
# Root directory of the cloned PDK repository
export PDK_ROOT=/path/to/SCCAD-3nm-PDK

# Cadence Front-side PnR base path
export PNR_ROOT="$PDK_ROOT/PDK-Cadence/Front-side Version/PnR"
```

```
# Technology LEF path (Innovus)
export TECH_LEF="$PNR_ROOT/TECH-LEF"

# Cell LEF path
export LEF_PATH="$PNR_ROOT/LEF"

# Liberty timing library path
export LIB_PATH="$PNR_ROOT/LIB_DB"

# QRC tech file path (Quantus)
export QRC_TECH="$PNR_ROOT/QRC"

# SPICE model path
export MODEL_PATH="$PDK_ROOT/PDK-Cadence/Front-side Version/PDK Development/MODEL"
```

Note: For the Back-side Version (BPR), replace "Front-side Version" with "Back-side Version" in the paths above.

Verify the key PDK files are accessible:

```
echo $PDK_ROOT
ls "$TECH_LEF"
ls "$LEF_PATH"
ls "$LIB_PATH"
ls "$QRC_TECH"
```

Required EDA Tools:

The following Cadence tools must be installed and accessible via your system PATH.

- **Cadence Innovus** — Primary tool for place-and-route. The **innovus** executable must be accessible.
- **Cadence Quantus** — Parasitic extraction tool. The **quantus** executable must be accessible.
- **Cadence Tempus** — Static timing analysis and sign-off tool. The **tempus** executable must be accessible.
- **Cadence Genus** — Logic synthesis tool. The **genus** executable must be accessible.

Item	Requirement
Operating System	RHEL 7/8, CentOS 7, or compatible Linux (64-bit)
Shell	bash or csh/tcsh
TCL	Version 8.5 or later
Python	Version 3.8 or later
Disk Space	Minimum 10 GB free
Memory	Minimum 32 GB RAM recommended

License Configuration: Set the Cadence license server address using **CDS_LIC_FILE**:

```
export CDS_LIC_FILE=5280@your.license.server
which innovus
which quantus
which tempus
```

3 Running PnR with Cadence Innovus

3.1 Essential PDK Files for PnR

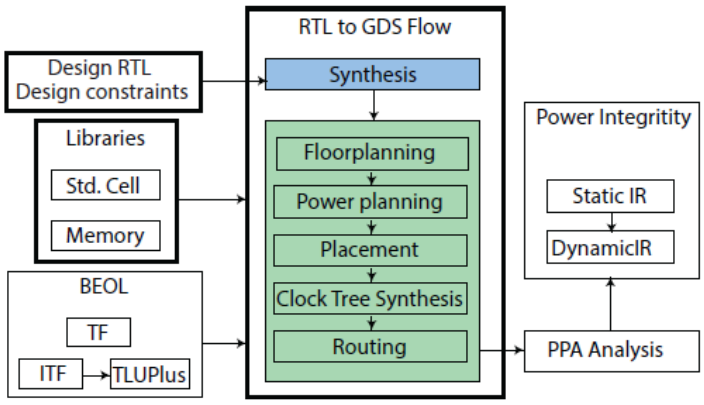
Before launching a PnR run, it is important to understand which PDK files are loaded at each stage of the Innovus flow. Unlike Synopsys ICC2, which uses the NDM format, Cadence Innovus loads technology LEF, cell LEF, and Liberty files directly. Parasitic extraction uses Quantus with a QRC technology file.

Directory / File	Format	Tool	Description
PDK-Cadence/Front-side Version/PnR/TECH-LEF/3nm_GAA_FSPR_tech.lef	.lef	Innovus	Technology LEF: layer definitions, via rules, routing directions
PDK-Cadence/Front-side Version/PnR/LEF/3nm_GAA_FSPR.lef	.lef	Innovus	Cell LEF: macro dimensions and pin geometry
PDK-Cadence/Front-side Version/PnR/LIB_DB/3nm_GAA_FSPR_lvt_nldm.lib	.lib	Innovus / Tempus	Liberty: timing arcs, power tables
PDK-Cadence/Front-side Version/PnR/QRC/3nm_GAA_FSPR.tch	.tch	Quantus	QRC technology file for parasitic extraction

The technology LEF and cell LEF define the physical design rules and cell abstracts loaded at Innovus startup. The **Liberty (.lib)** file provides timing and power data for placement optimization and sign-off with Tempus. The **QRC technology file** is the Quantus-native RC model used for full-chip parasitic extraction after routing.

3.2 PnR Flow Overview

The SCCAD-3nm PDK supports a complete RTL-to-GDS back-end flow using Cadence Innovus. The overall flow is illustrated in **Figure 3**, spanning from RTL synthesis through physical implementation and PPA analysis. Each stage is described in detail below.



[Figure3. Overall PnR Flow used in this work]

Step 1 — RTL Synthesis

The flow begins with logic synthesis using **Cadence Genus**. The user provides RTL source files and SDC constraints specifying target clock frequency and I/O delays. Genus maps the RTL onto the SCCAD-3nm standard-cell library and produces a gate-level netlist and synthesized SDC, which are passed to Innovus.

```
genus -batch -files scripts/genus/run_syn.tcl \
      -log logs/syn.log
```

Step 2 — Design Setup in Innovus

Innovus loads design data using the **read_physical** and **read_timing** commands. Unlike ICC2's NDM, Innovus reads technology LEF, cell LEF, and Liberty files directly at startup.

```
# Load technology and cell LEF
read_physical -lef [list \
    "$TECH_LEF/3nm_GAA_FSPR_tech.lef" \
    "$LEF_PATH/3nm_GAA_FSPR.lef"]

# Load timing library
read_timing -lib "$LIB_PATH/3nm_GAA_FSPR_lvt_nldm.lib"

# Load netlist and constraints
read_netlist  results/syn/netlist.v
read_sdc      results/syn/constraints.sdc
init_design
```

Step 3 — Floorplanning and Power Planning

Floorplanning defines the die area and core utilization. Power planning creates the VDD/VSS power mesh. For BPR-based designs, the buried power rails are pre-defined in the cell LEF; front-side power rings and stripes are added on top.

```
# Initialize floorplan
floorPlan -coreMarginsBy die \
          -site          FreePDK45_38x28_10R_NP_162NW_34O \
          -coreSpacings  2 2 2 2

# Add power domains
add_power_domains -primary VDD -ground VSS

# Create power mesh on upper metal layers
create_pg_mesh_pattern -layers {M4 M5} \
                       -nets   {VDD VSS}
compile_pg
```

Step 4 — Placement

Innovus performs timing-driven placement using **place_design**, followed by post-placement optimization with **optDesign -preCTS**. The tool uses ideal clock assumptions at this stage.

```
place_design
optDesign -preCTS
```

Step 5 — Clock Tree Synthesis (CTS)

Clock Tree Synthesis is performed using **ccopt_design**, Innovus's constraint-driven CTS engine. NDR rules from the PDK technology file are applied for clock net shielding and spacing. Post-CTS optimization follows immediately.

```
# Create CTS spec from SDC clocks
create_ccopt_clock_tree_spec

# Run CTS
ccopt_design

# Post-CTS optimization
optDesign -postCTS
```

Step 6 — Routing

Global and detail routing are performed by **route_design**, followed by post-route optimization with **optDesign -postRoute**. The router uses the metal stack and spacing rules from the technology LEF.

```
route_design
optDesign -postRoute
```

Step 7 — PPA Analysis

After routing, the final Power, Performance, and Area (PPA) metrics are evaluated. **Performance** is assessed through static timing analysis using **Cadence Tempus**, reporting worst negative slack (WNS) and total negative slack (TNS). **Power** analysis uses switching activity from simulation. **Area** is reported directly from Innovus after routing.

```
# Static timing analysis — Tempus
tempus -f scripts/tempus/run_sta.tcl \
       -log logs/sta.log

# Area report — Innovus
report_design_area
report_utilization
```

Upon successful sign-off, stream out the final GDS from Innovus:

```
streamOut results/final/design.gds \
          -mapFile scripts/gds/layermap.map \
          -libName my_design
```

4 Troubleshooting

This chapter describes common issues encountered when setting up and running the SCCAD-3nm PDK with Cadence Innovus, along with recommended solutions.

4.1 Installation and Environment Issues

PDK_ROOT not set or files not found

Symptom: ls $TECH_LEF returns empty results or "No such file or directory".

Solution: Verify that PDK_ROOT is exported and points to the repository root:

```
echo $PDK_ROOT
source ~/.bashrc
ls $PDK_ROOT/TECH-LEF
```

License server error on tool startup

Symptom: Innovus or Tempus exits with "license checkout failed".

Solution: Verify CDS_LIC_FILE and check network connectivity:

```
echo $CDS_LIC_FILE
ping your.license.server
lmstat -a -c $CDS_LIC_FILE
```

4.2 Innovus Setup Issues

read_physical fails to load LEF files

Symptom: Innovus reports "cannot open LEF file" or layer names are missing after read_physical.

Solution: Verify both the technology LEF and cell LEF are included in the read_physical command, in the correct order (tech LEF first):

```
read_physical -lef [list \
    $TECH_LEF/3nm_GAA_FSPR_tech.lef \
    $LEF_PATH/3nm_GAA_FSPR.lef]

# Verify layers were loaded correctly
get_db tech.layers.name
```

Liberty version mismatch warning

Symptom: Innovus prints warnings about missing timing arcs after read_timing.

Solution: Ensure the correct .lib file is loaded and that it was characterized for the target corner. Verify with:

```
read_timing -lib $LIB_PATH/3nm_GAA_FSPR_lvt_nldm.lib

# Check loaded libraries
report_timing_setting
```

4.3 PnR Flow Issues

Placement congestion / utilization too high

Symptom: place_design reports congestion overflow or routing fails to complete.

Solution: Reduce core utilization:

```
# Reduce from default to 0.65
floorPlan -coreMarginsBy die \
          -site FreePDK45_38x28_10R_NP_162NW_34O \
          -coreSpacings 2 2 2 2 \
          -coreDensity 0.65
```

Timing not closed after optDesign -postRoute

Symptom: Tempus reports negative WNS even after multiple optimization iterations.

Solution: Check SDC constraints and verify the correct library corner is active:

```
# In Tempus, report worst timing path
report_timing -max_paths 10

# Check active corners
report_constraint -all_violators
```

Quantus extraction fails or empty SPEF

Symptom: Quantus exits with an error or the resulting SPEF has no parasitics.

Solution: Verify the QRC tech file path and the routed DEF file are correctly specified in the Quantus config:

```
# Check Quantus config file
cat scripts/quantus/quantus.conf

# Key fields to verify:
# qrc_tech_file: path to .tch file
# design_cdl:    path to CDL netlist
# input_db:      path to routed DEF
```

References

[1] J.-S. Yoon, J. Jeong, S. Lee, J. Lee, S. Lee, R.-H. Baek, and S. K. Lim, "Performance, Power, and Area of Standard Cells in Sub 3 nm Node Using Buried Power Rail," *IEEE Transactions on Electron Devices*, vol. 69, no. 3, pp. 894–899, Mar. 2022. doi: 10.1109/TED.2021.3138865

[2] S. M. Shaji, L. Zhu, J. Yoon, and S. K. Lim, "A Comparative Study on Front-Side, Buried and Back-Side Power Rail Topologies in 3nm Technology Node," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, 2023. doi: 10.1109/ISLPED58423.2023.10244504

[3] S. Sadangi, V. Pasumarthy, S. Pitts, and W. R. Davis, “FreePDK3: A Novel PDK for Physical Verification at the 3nm Node,” Master’s Thesis, NC State University, 2021.

[4] D. E. Shim, P. Kumar, A. A. Kini, M. Mallikarjuna, M. N. H. Shazon, and A. Naeemi, “GT3: An Open-Source 3-nm GAAFET PDK and Platform for End-to-End Evaluation of Emerging Technologies,” *IEEE Transactions on Electron Devices*, doi: 10.1109/TED.2025.3540760, 2025.